

AI KAVACH 2026 — DATA-CENTRIC INNOVATION CHALLENGE

AstraCRS

An AI Assistant That Finds Security Bugs in Software, Fixes Them, and Proves the Fix Works

PROJECT ABSTRACT

01 Problem Statement & Motivation

Military and defence software systems are high-value targets for cyber adversaries. A single undetected vulnerability in a communication system, command-and-control platform, or logistics tool can become an entry point for espionage, sabotage, or disruption of operations. The consequences are not theoretical — they are a matter of national security.

Today, identifying and patching these vulnerabilities is a heavily manual process. It requires scarce specialist talent, often takes weeks of effort per software system, and produces results that are difficult for non-technical decision-makers to act upon. In operational scenarios, the armed forces may not have the luxury of that time.

The core challenge: Build a cyber-reasoning system — powered by AI and integrated with automated security tools — that can autonomously discover a vulnerability in software, generate a fix, and prove that the fix holds, all without human intervention.

02 Proposed Solution

AstraCRS is a lightweight, fully autonomous AI-powered security assistant that performs the complete vulnerability lifecycle — from discovery to verified repair — in a single automated pipeline. It combines proven security testing tools with modern AI code understanding to deliver a system that is practical, trustworthy, and field-deployable.

The system operates in four core stages, each feeding into the next without requiring human involvement:



STAGE	WHAT HAPPENS	SIMPLE ANALOGY
Scan	Reads through the software's source code and flags areas that look risky or suspicious — functions that handle external data, memory operations, input parsing.	Like a spell-checker, but for security flaws in code.
Stress-Test	Bombards those flagged areas with thousands of crafted and random inputs to see if any of them cause the software to crash or behave dangerously.	Like crash-testing a car at high speed to find the weak points.
AI Fix	When a real crash is confirmed, an AI code model analyses the crash location, understands the root cause, and writes a small, precise code repair.	Like a mechanic diagnosing the failure and writing a targeted repair instruction.
Verify	Replays the exact attack that caused the crash to confirm it no longer works. Then runs all existing software tests to ensure nothing else has broken.	Like re-crash-testing the repaired car, plus a full safety inspection.

Key principle: The system only reports *confirmed, reproducible* vulnerabilities. If it says something is broken, it can show you the exact input that causes the crash. Zero guesswork, zero false alarms.

03 What Makes AstraCRS Different

The obvious approach to this challenge — connect a bug-finder to an AI model — is what most solutions will attempt. AstraCRS goes further in two critical ways that are built into the core system, not afterthoughts.

A. Danger-Level Triage — Prioritising by Actual Threat

Not all bugs are equally dangerous. A crash in a logging function is an inconvenience. A crash in a network message handler is a national security risk — an adversary could exploit it remotely without physical access to the system.

AstraCRS automatically classifies every discovered vulnerability by how dangerous it actually is in a real-world attack scenario:

DANGER LEVEL	WHAT IT MEANS
CRITICAL	Reachable from the network by an external attacker. Could allow remote system takeover or data exfiltration.
HIGH	Could crash the system when processing external data (files, messages). Denial-of-service risk.
MEDIUM / LOW	Requires local access or unlikely conditions to trigger. Still fixed, but lower priority.

This means decision-makers receive a **prioritised action list** — "fix these two first, they're the ones an adversary could actually exploit from outside" — instead of an overwhelming, unsorted wall of technical bug reports.

B. Officer-Readable Threat Reports

Every existing security tool produces output meant for software engineers. But in a defence context, the person who needs to make the decision — "Is this system safe to deploy?" — is often a commanding officer, not a programmer.

For every vulnerability it finds and fixes, AstraCRS generates a plain-language threat summary that any decision-maker can read and act on:



Module:	Message handling subsystem
What was found:	A flaw that allows an attacker on the same network to send a specially crafted message that crashes the system.
Danger Level:	CRITICAL — remotely exploitable
Fix Applied:	Input size validation added to message parser.
Verification:	✓ Attack replayed — crash no longer occurs. All system tests pass.

A Colonel can read that. A General can act on it. No translator needed.

04 Technology Stack

CODE SCANNING

Semgrep, Tree-sitter

Open-source security scanning tools

STRESS TESTING

AFL++, AddressSanitizer

Industry-standard automated crash finder

AI CODE UNDERSTANDING

Ollama + Open-source AI Models

Runs entirely offline on a laptop GPU

VERIFICATION

Crash Replay + Test Harness

Automated pass/fail validation

ORCHESTRATION

Python

Connects all components together

DEPLOYMENT

Docker

Packages entire system into one portable unit

Hardware requirement: A standard laptop with a mid-range GPU (8 GB VRAM). No servers, no cloud subscriptions, no internet connectivity required at any point.

05 Why This Innovation Is Possible Now

The individual building blocks — code scanners, stress-testing tools, automated test frameworks — have existed for years. The military already uses many of them. So what's new?

The missing piece was always the middle step. A scanner could flag a suspicious line. A stress-tester could prove the crash. But *understanding why the code was broken and writing a correct fix* — that required a

human expert sitting in the loop, reading crash reports, reasoning about root causes, and manually writing repairs. That bottleneck made full automation impossible.

AI models capable of genuinely reading, understanding, and writing code at a useful level only became available in 2024–2025. For the first time, we can close the loop from "find the bug" to "prove the fix" without a human in the middle. That is the core innovation — not any single tool, but the ability to **connect them into a fully autonomous pipeline** that wasn't reliably possible even two years ago.

06 Future Capabilities — Finale Integration Roadmap

The core system (scan → fix → verify + danger triage + readable reports) will be functional at the qualifier stage. During the 36-hour Grand Finale, we plan to integrate three additional capabilities that significantly expand the system's power:

STATUS	CAPABILITY	WHAT IT DOES	WHY IT MATTERS
CORE	Scan → Fix → Verify Pipeline	Autonomous end-to-end vulnerability lifecycle.	The foundational capability. Everything else builds on this.
CORE	Danger-Level Triage	Classifies each bug by real-world exploitability.	Turns raw findings into prioritised military intelligence.
CORE	Officer-Readable Reports	Plain-language summaries of threats and fixes.	Enables non-technical decision-makers to act immediately.
FINALE	Fix Re-Stress-Testing	After applying a patch, the system attacks the repaired area again with thousands of <i>new</i> inputs to ensure the fix is truly robust — not just a band-aid that hides the symptom.	Prevents superficial patches. Guarantees the repair holds against attacks the original test didn't cover.
FINALE	Binary-Level Analysis	Works on compiled software even when the original source code is not available, by reverse-engineering the binary first.	Critical for real military use — many systems use proprietary/vendor software where source code is unavailable.

STATUS	CAPABILITY	WHAT IT DOES	WHY IT MATTERS
FINALE	Pattern Learning Memory	The system remembers vulnerability patterns from past scans. After analysing multiple projects, it recognises recurring mistake patterns and checks for them proactively.	Like a doctor who gets better with experience — the system improves the more software it analyses.

07 Deliverables & Performance Targets

QUALIFIER DELIVERABLES - Working prototype of the full scan → fix → verify cycle - Danger-level classification for every discovered vulnerability - Auto-generated plain-language threat reports - Live demonstration on a sample vulnerable program	FINALE DELIVERABLES - Enhanced system with fix re-testing, binary support, and learning memory - Web dashboard showing real-time pipeline progress - Testing against the simulated Armed Forces software environment - Complete documentation and deployment guide
---	---

PERFORMANCE METRIC	TARGET
Time from scan to verified fix	Under 5 minutes per confirmed vulnerability
Hardware required	Single laptop — no servers, no cloud infrastructure
False positive rate	0% — every reported bug is a proven, reproducible crash
Internet dependency	None — fully offline, air-gap compatible

PERFORMANCE METRIC	TARGET
Report readability	Understandable by a non-technical commanding officer

08 Summary

AstraCRS is a fully autonomous, offline-capable AI security assistant that finds real vulnerabilities in software, fixes them, proves the fix works, prioritises threats by danger level, and explains everything in plain language that any decision-maker can act on. It runs on a single laptop, needs no internet, and produces zero false alarms.

In one line: It's like having a tireless security expert who finds real bugs, fixes them, proves the fix, ranks the danger, writes you a clear report — and fits in a backpack.